Programme de formation — AI/ML Engineering

Objectif général : former des profils capables de préparer des données, développer et évaluer des modèles de Machine Learning, construire des applications d'IA générative et mettre ces solutions en production.

Le parcours suit une progression logique : Python et données → Machine Learning → Deep Learning et NLP → LLM et applications IA → automatisation → déploiement et MLOps.

Module 1 — Python pour l'IA et le Machine Learning

Objectif : acquérir les bases de programmation nécessaires pour manipuler des données et développer des solutions d'IA.

Ce qu'il faut comprendre et apprendre

- Les bases de Python : variables, types de données, conditions, boucles, fonctions et exceptions.
- Les structures de données : listes, dictionnaires, tuples et ensembles.
- La programmation modulaire, les modules, les packages et les notions essentielles de programmation orientée objet.
- La manipulation de tableaux numériques avec NumPy.
- La manipulation de données tabulaires avec Pandas.
- La lecture et l'écriture de fichiers CSV, JSON et Excel.
- L'utilisation des notebooks Jupyter et la création de scripts Python.
- La gestion des environnements virtuels et des dépendances.

Outils principaux : Python, NumPy, Pandas, Jupyter Notebook.

Résultat attendu : écrire des scripts Python capables de charger, manipuler, analyser et exporter des données.

Module 2 — Préparation et analyse des données

Objectif : transformer des données brutes en données fiables et adaptées à l'entraînement des modèles.

Ce qu'il faut comprendre et apprendre

- Comprendre les types de données, les distributions et les statistiques descriptives.
- Réaliser une analyse exploratoire des données (EDA).
- Créer des visualisations pour comprendre les tendances et les relations.
- Traiter les valeurs manquantes, les doublons, les incohérences et les valeurs aberrantes.
- Encoder les variables catégorielles et normaliser ou standardiser les variables numériques.
- Réaliser du feature engineering : créer et sélectionner des variables utiles.

- Séparer les données en ensembles d’entraînement, de validation et de test.
- Éviter les fuites de données (data leakage).
- Construire des pipelines de préparation reproductibles.

Outils principaux : Pandas, NumPy, Matplotlib, Seaborn, scikit-learn.

Résultat attendu : produire un jeu de données préparé, documenté et utilisable dans un pipeline de Machine Learning.

Module 3 — Machine Learning

Objectif : apprendre à sélectionner, entraîner et utiliser des modèles capables de faire des prédictions à partir des données.

Ce qu’il faut comprendre et apprendre

1. Les grandes familles du Machine Learning

- Apprentissage supervisé et non supervisé.
- Différence entre classification, régression et clustering.
- Notions de variables d’entrée, de cible, d’entraînement et de prédiction.

2. Les principaux algorithmes

- Régression linéaire et régression logistique.
- Arbres de décision et Random Forest.
- Support Vector Machines (SVM).
- Gradient Boosting, avec une introduction à XGBoost ou CatBoost.
- K-means pour le clustering.

3. La mise en pratique

- Construire un modèle simple servant de référence (baseline).
- Entraîner différents modèles sur un même jeu de données.
- Effectuer des prédictions sur de nouvelles données.
- Comparer les premières performances et comprendre les forces et limites des algorithmes.

Outils principaux : scikit-learn, avec une introduction à XGBoost ou CatBoost.

Résultat attendu : développer un premier modèle de Machine Learning et justifier le choix de l’algorithme utilisé.

Module 4 — Évaluation et amélioration des modèles

Objectif : mesurer correctement les performances, identifier les erreurs et améliorer les modèles de manière rigoureuse.

Ce qu'il faut comprendre et apprendre

- Les métriques de classification : précision, rappel, F1-score, matrice de confusion, ROC-AUC et PR-AUC.
- Les métriques de régression : MAE, MSE et RMSE.
- La validation croisée et le choix d'une stratégie de validation adaptée aux données.
- Le surapprentissage (overfitting) et le sous-apprentissage (underfitting).
- Le biais, la variance et la généralisation.
- La recherche des meilleurs hyperparamètres avec GridSearchCV, RandomizedSearchCV ou Optuna.
- La sélection des variables et la comparaison des modèles.
- L'analyse des erreurs et le choix du seuil de classification.
- L'interprétabilité des résultats et l'importance des variables.
- Une introduction à SHAP pour mieux comprendre certaines prédictions.
- La prévention des fuites de données pendant la validation et l'optimisation.

Outils principaux : scikit-learn, Optuna, SHAP.

Résultat attendu : produire un benchmark de plusieurs modèles, comparer leurs performances, expliquer leurs erreurs et justifier le modèle retenu.

Module 5 — Deep Learning

Objectif : comprendre et entraîner des réseaux de neurones pour résoudre des problèmes plus complexes.

Ce qu'il faut comprendre et apprendre

- Le fonctionnement d'un neurone artificiel et d'un réseau de neurones.
- Les tenseurs et les opérations différentiables.
- Les fonctions d'activation et les fonctions de perte.
- La descente de gradient, la rétropropagation et les optimiseurs.
- La construction de modèles, les boucles d'entraînement et de validation.
- La régularisation, le dropout, la normalisation et le réglage du taux d'apprentissage.
- L'analyse des courbes d'apprentissage et le diagnostic du surapprentissage.
- Les réseaux entièrement connectés (MLP).
- Les réseaux convolutifs (CNN) pour les images.
- Une introduction aux réseaux séquentiels, notamment RNN et LSTM.
- Le transfert d'apprentissage pour exploiter des modèles préentraînés.

Outils principaux : PyTorch.

Résultat attendu : entraîner un réseau de neurones, comparer ses résultats à une approche de référence et analyser ses performances.

Module 6 — NLP : traitement du langage naturel

Objectif : apprendre à traiter automatiquement du texte et à construire des modèles spécialisés dans les tâches linguistiques.

Ce qu'il faut comprendre et apprendre

- Les particularités des données textuelles et les étapes de prétraitement.
- Le nettoyage, la normalisation et la tokenisation des textes.
- Les notions de mots, tokens, vocabulaire et représentation numérique du texte.
- Les approches classiques : Bag of Words et TF-IDF.
- La classification de textes, l'analyse de sentiments et la catégorisation de documents.
- L'extraction d'informations et l'identification de mots ou d'entités.
- La préparation des données textuelles pour l'entraînement d'un modèle.
- L'évaluation des modèles NLP et l'analyse des erreurs.
- Une introduction aux Transformers et aux modèles de langage préentraînés.

Outils principaux : Python, scikit-learn, spaCy, NLTK et Hugging Face.

Résultat attendu : construire un modèle capable de réaliser une tâche NLP, par exemple classer des messages ou analyser des avis clients.

Module 7 — LLM et IA générative

Objectif : comprendre les grands modèles de langage et apprendre à les exploiter dans des applications concrètes.

Ce qu'il faut comprendre et apprendre

- Les principes généraux du fonctionnement des Large Language Models (LLM).
- Les tokens, les embeddings, la fenêtre de contexte et la génération de texte.
- Les grandes étapes de l'entraînement et de l'utilisation d'un modèle de langage.
- Les différences entre modèles accessibles par API et modèles ouverts exécutables dans un environnement contrôlé.
- Les modèles génératifs pour le texte et une introduction aux autres modalités.
- Les paramètres de génération, notamment la température et les limites de sortie.
- Les hallucinations, biais, erreurs factuelles et limites des modèles.
- Les enjeux de confidentialité, de coût, de latence et de choix du modèle.
- Les cas d'usage : résumé, extraction d'information, assistance, classification et génération.

Outils principaux : Hugging Face et API de modèles d'IA.

Résultat attendu : sélectionner un modèle adapté à un besoin et expliquer ses capacités, ses limites et les coûts associés à son utilisation.

Module 8 — Prompt Engineering et API de modèles d'IA

Objectif : communiquer efficacement avec les modèles et les intégrer dans des applications Python.

Ce qu'il faut comprendre et apprendre

- Construire des prompts clairs avec des instructions et un contexte précis.
- Structurer les messages système et utilisateur.
- Utiliser des exemples pour orienter les réponses (few-shot prompting).
- Définir des contraintes de format, de contenu et de longueur.
- Obtenir des sorties structurées, notamment au format JSON.
- Appeler une API de modèle depuis Python.
- Gérer les clés API, les variables d'environnement et les secrets.
- Gérer les erreurs, délais d'attente, limites de débit et tentatives de reprise.
- Comprendre les coûts liés aux tokens et mesurer la qualité des réponses.
- Tester et comparer différentes formulations de prompts.

Outils principaux : Python, API de modèles d'IA, Pydantic pour valider les sorties structurées.

Résultat attendu : développer une application Python qui utilise un modèle via API et gère proprement les entrées, les réponses et les erreurs.

Module 9 — Embeddings et recherche sémantique

Objectif : comprendre comment représenter le sens du texte sous forme de vecteurs et rechercher des contenus sémantiquement proches.

Ce qu'il faut comprendre et apprendre

- La différence entre une représentation textuelle classique et un embedding.
- La transformation d'un texte en vecteur numérique.
- Les notions de similarité cosinus, distance et proximité vectorielle.
- La génération et l'utilisation d'embeddings préentraînés.
- L'indexation et la recherche de vecteurs.
- Les métadonnées et le filtrage des résultats.
- La recherche sémantique et ses différences avec la recherche par mots-clés.
- Les notions de pertinence, de précision et de rappel dans les résultats de recherche.

Outils principaux : modèles d'embeddings Hugging Face, FAISS ou Chroma.

Résultat attendu : développer un moteur de recherche sémantique qui retrouve les textes les plus pertinents par rapport à une question.

Module 10 — RAG : génération augmentée par récupération

Objectif : construire des applications qui s'appuient sur des documents ou des sources externes pour fournir des réponses mieux contextualisées.

Ce qu'il faut comprendre et apprendre

- Le principe du RAG et les problèmes auxquels il répond.
- La collecte et l'extraction de contenu depuis des fichiers PDF, DOCX, TXT ou des sources structurées.
- Le découpage des documents en segments (chunking).
- La création d'embeddings et l'indexation des segments.
- La récupération des passages pertinents et la constitution du contexte.
- L'envoi du contexte au LLM pour générer une réponse.
- La gestion des métadonnées et des références aux sources.
- La recherche hybride et le reranking, en approfondissement.
- La gestion des questions auxquelles les documents ne permettent pas de répondre.
- L'évaluation séparée de la qualité de la recherche et de la génération.
- L'analyse des erreurs : contexte manquant, mauvais passages, réponses non fondées.

Outils principaux : Python, FAISS ou Chroma, API de LLM ou modèle ouvert, Hugging Face.

Résultat attendu : créer un assistant documentaire RAG qui répond à partir d'un corpus réel, cite ses sources et fait l'objet d'une évaluation sur un jeu de questions.

Module 11 — Chatbots et assistants IA

Objectif : transformer les capacités des modèles et du RAG en assistants conversationnels utilisables par des personnes.

Ce qu'il faut comprendre et apprendre

- L'architecture d'un chatbot et le traitement d'une conversation.
- La gestion de l'historique et du contexte conversationnel.
- La différence entre contexte envoyé au modèle et mémoire persistante.
- La gestion des sessions et des conversations à plusieurs échanges.
- L'intégration d'un LLM avec un moteur RAG.
- La validation des entrées et la gestion des erreurs de conversation.
- La conception des réponses et la gestion des situations où le modèle ne sait pas répondre.
- La création d'une interface de conversation.

- La protection des informations sensibles et la limitation des accès.
- L'évaluation de la pertinence, de la cohérence et de l'utilité des réponses.

Outils principaux : Python, API de modèles d'IA, FastAPI pour l'interface de service et Streamlit ou Gradio pour un prototype conversationnel.

Résultat attendu : développer un chatbot fonctionnel, capable de conserver le contexte de la conversation et, si nécessaire, d'interroger une base documentaire.

Module 12 — Automatisation et agents IA

Objectif : développer des systèmes d'IA capables d'utiliser des outils et d'exécuter des tâches composées de plusieurs étapes.

Ce qu'il faut comprendre et apprendre

- La différence entre une automatisation classique, un workflow IA et un agent.
- Le fonctionnement des appels de fonctions et l'utilisation d'outils externes.
- La conception de workflows avec plusieurs étapes.
- La planification des actions, la gestion de l'état et le contrôle de l'exécution.
- L'intégration d'API, de bases de données et de fonctions Python.
- La gestion des erreurs et les mécanismes de reprise.
- Les limites d'exécution, les coûts et les conditions d'arrêt.
- Les contrôles d'accès et la validation des actions sensibles.
- Les risques d'injection de prompt et d'utilisation abusive des outils.
- L'évaluation de la fiabilité d'un agent, de ses décisions et des résultats produits.
- Les cas où un workflow déterministe est préférable à un agent autonome.

Outils principaux : Python, API de LLM et, en approfondissement, LangGraph ou un framework d'agents adapté.

Résultat attendu : construire un agent ou un workflow IA qui utilise au moins un outil, exécute une tâche en plusieurs étapes et comporte des limites et des contrôles explicites.

Module 13 — Déploiement et MLOps

Objectif : rendre les modèles et applications d'IA accessibles, testables, reproductibles et maintenables en production.

Ce qu'il faut comprendre et apprendre

1. Exposition des modèles

- Créer des API avec FastAPI.
- Valider les entrées et les sorties avec Pydantic.
- Charger un modèle de Machine Learning et exposer une fonction de prédiction.

- Exposer une application LLM ou RAG par une API.

2. Conteneurisation et tests

- Construire des images avec Docker et comprendre Docker Compose.
- Gérer la configuration, les variables d'environnement et les secrets.
- Écrire des tests unitaires et d'intégration avec pytest.
- Vérifier les entrées, les sorties, les erreurs et les réponses de l'API.

3. Suivi des expérimentations et versionnement

- Suivre les expériences, les paramètres et les métriques avec MLflow.
- Enregistrer les modèles et comparer différentes versions.
- Comprendre le versionnement des données et des artefacts, avec une introduction à DVC.
- Utiliser Git et GitHub pour versionner le code.

4. Automatisation et exploitation

- Automatiser les tests avec GitHub Actions.
- Déployer une API ou une application IA dans un environnement adapté.
- Surveiller la latence, les erreurs, les coûts et la disponibilité.
- Suivre la dérive des données et l'évolution des performances.
- Documenter les procédures de reproduction, de mise à jour et de retour arrière.

Outils principaux : FastAPI, Pydantic, Docker, pytest, MLflow, Git, GitHub Actions et notions de DVC.

Résultat attendu : livrer un service IA testé, conteneurisé et déployé, avec un suivi des expériences et une procédure de maintenance.

Projet final — Construire une solution complète d'IA

Le projet final doit réunir les compétences acquises dans le parcours.

Le participant devra :

1. Définir un problème concret, les utilisateurs concernés et le résultat attendu.
2. Collecter, préparer et analyser les données.
3. Choisir une approche justifiée : Machine Learning classique, Deep Learning ou LLM.
4. Entraîner ou intégrer le modèle, puis évaluer ses performances.
5. Analyser les erreurs et documenter les limites de la solution.
6. Développer une API ou une interface adaptée.
7. Si le projet porte sur un assistant documentaire, intégrer un RAG et évaluer la qualité des réponses.

8. Automatiser les tests, conteneuriser l'application et préparer son déploiement.
9. Documenter le projet, ses résultats et les choix techniques, puis effectuer une démonstration.

Livrable final : un projet présentable sur GitHub, reproductible et testé, avec des résultats mesurés, une documentation claire et une démonstration fonctionnelle.

Méthode de travail recommandée

Pour éviter d'accumuler des notions sans savoir les appliquer, chaque module suit la même démarche :

- Comprendre : assimiler les concepts et le fonctionnement des méthodes.
- Pratiquer : réaliser des exercices ciblés et manipuler les outils.
- Expérimenter : comparer les approches, analyser les erreurs et justifier les choix.
- Livrer : produire un mini-projet testé, documenté et reproductible.

Le parcours peut être organisé sur 12 semaines en format intensif, comme le propose ton document, à raison d'environ 15 à 20 heures de travail par semaine. Cela demande toutefois de sélectionner les notions prioritaires et de traiter certains sujets en introduction ou en approfondissement, plutôt que de viser une maîtrise complète de tous les outils.

Le principe central est de former des profils capables de développer, évaluer et livrer des solutions d'IA fiables, plutôt que de simplement savoir appeler une API de modèle ou exécuter un notebook.